TALLINN UNIVERSITY OF TECHNOLOGY

School of Information Technologies

Héctor Barrios Luján — 266034IV

Secure Password Manager Application

Web Application Security - Checkpoint 1

Project Design Documentation

Supervisor: Ali Ghasempour

Tallinn 2026

1.INTRODUCTION

In the modern digital landscape, the exponential growth of online services has forced individuals to manage dozens, or even hundreds, of distinct digital credentials. Consequently, the reliance on weak, reused, or improperly managed passwords has become one of the most critical human-factor vulnerabilities in cybersecurity. While commercial password managers offer a robust solution to this issue, proprietary software often lacks transparency regarding its internal architecture and data handling processes. This academic project, developed as part of the Web Application Security course at Tallinn University of Technology (TalTech), aims to bridge this gap by designing, implementing, and securing a fully functional, open-source web-based password manager.

The primary objective of this project is to apply theoretical security concepts—such as secure authentication, robust cryptographic data storage, and input validation—into a real-world, hands-on software development lifecycle. Rather than focusing solely on superficial functionality, the core philosophy of this work is grounded in a "shift-left" security approach, where threat modeling, secure architectural planning, and vulnerability mitigation are prioritized from the very beginning of the development process.

Throughout this project, the application will be subjected to rigorous self-assessment and simulated penetration testing to identify and remediate potential security flaws, including broken access controls, injection risks, and improper exception handling. By undertaking this endeavor, the project not only delivers a secure and practical tool for credential management but also serves as a comprehensive practical exercise in translating academic security principles into resilient software engineering practices.

2. Project Requirements and Scope

To ensure a structured development process and a clearly defined boundary for the secure password manager, this section outlines the core functional requirements, technical scope, and security objectives of the application.

2.1 Functional Requirements

The primary goal of the system is to provide a user-friendly yet highly secure web interface for managing sensitive credentials. The functional scope includes:

- **User Authentication & Session Management:** Implementation of a robust registration and login mechanism, ensuring that user sessions are handled securely to prevent session hijacking or unauthorized access.

- **Encrypted Vault Storage:** Users must be able to securely store, retrieve, and update their credentials (such as website URLs, usernames, and passwords) within a digital vault.
- **Database Integrity:** The backend database must remain resilient against structural corruption or unauthorized privilege escalation (e.g., preventing an attacker from trivially modifying user roles, such as setting `admin = 1`).
- **Error Handling & Feedback:** The application must handle exceptional conditions gracefully without leaking sensitive system information or stack traces to potential attackers.

2.2 Security Scope and Defensive Objectives

In alignment with the core principles of the Web Application Security course, the development of this password manager will actively incorporate and defend against top web application vulnerabilities, specifically addressing:

- **Broken Access Control:** Ensuring strict authorization checks so that users can only access and modify their own stored credentials.
- **Input and Output Validation:** Implementing rigorous whitelisting mechanisms for all user-supplied data to neutralize potential injection vectors (such as SQL injection or XSS).
- **Cryptographic Protection:** Utilizing secure, modern cryptographic standards (such as TLS for data in transit and robust hashing/encryption algorithms for data at rest) to protect sensitive vault contents.
- **Logging and Monitoring:** Integrating activity logs to trace system events and detect potential malicious interactions or anomalies.

By limiting the scope to a focused, self-contained web application rather than an enterprise-scale solution, this project allows for a deep, granular analysis of secure coding practices and vulnerability mitigation throughout the software lifecycle.

3. Threat Modeling and Actors

A comprehensive security analysis requires a clear understanding of who interacts with the system, what assets are at risk, and what potential adversaries might attempt to compromise. This section defines the primary actors within the secure password manager ecosystem and outlines the foundational threat model.

3.1 System Actors

To properly delineate access levels and security boundaries, the application defines two primary actors:

1. **Legitimate User:** The authenticated owner of the password vault. This actor interacts with the web interface to store, retrieve, update, and delete their encrypted credentials securely. The system must guarantee that this user can only access their own data.
2. **Malicious Adversary (Attacker):** An external or internal hostile entity attempting to compromise the confidentiality, integrity, or availability of the system. This actor may lack valid credentials and attempts to bypass authentication, exploit input validation flaws, or manipulate backend databases.

Figure 1 illustrates the interaction boundaries and the high-level relationship between the system actors and the core functionalities of the password manager.

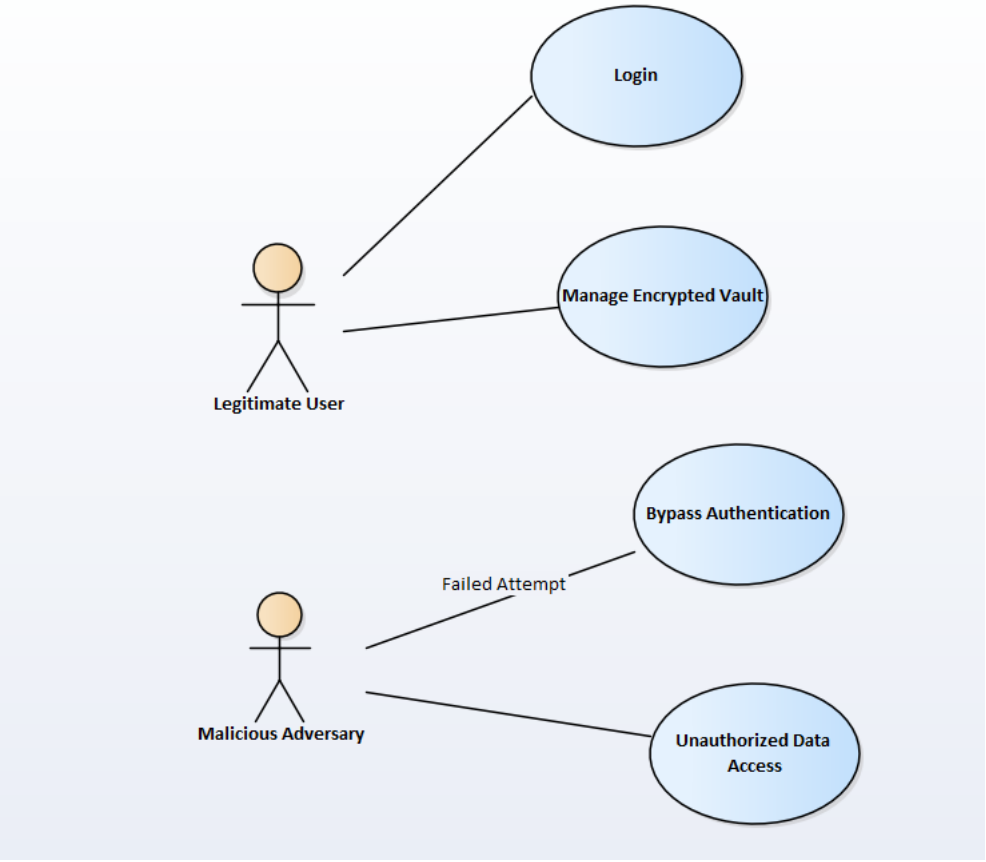


Figure 1. High-level actor interaction and system boundaries for the secure password manager.

3.2 Threat Modeling and Security Objectives

Following a proactive security approach, the threat model evaluates potential attack vectors targeting the application's core assets (specifically, the encrypted credential database and active user sessions). Based on common web vulnerabilities and the OWASP framework, the primary threats identified for this project include:

- **Credential Theft via Network Eavesdropping:** Interception of traffic between the browser and the server. *Mitigation objective:* Enforcement of mandatory TLS encryption.
- **Unauthorized Privilege Escalation / Broken Access Control:** An attacker manipulating requests or database parameters (e.g., altering user identification tokens) to access foreign vaults. *Mitigation objective:* Strict server-side authorization checks and robust session validation.
- **Database Tampering / Injection:** Attempts to inject malicious payloads (such as SQL injection) to corrupt or exfiltrate the credential storage. *Mitigation objective:* Input whitelisting and parameterized queries.
- **Brute-Force Attacks on Master Authentication:** Repeated automated attempts to guess the user's master password. *Mitigation objective:* Proper rate limiting and secure password hashing algorithms (e.g., Argon2 or bcrypt).

4. System Architecture and Design

To build a robust, maintainable, and secure web-based password manager, the application adopts a classic **Three-Tier Architectural Pattern**. This design strictly separates the user interface, the core business logic, and the persistent data storage into distinct logical layers. Such segregation is a fundamental security best practice, as it minimizes the attack surface, limits lateral movement in the event of a breach, and ensures that sensitive operations—such as cryptographic processing and credential retrieval—are handled securely on the server side rather than exposed to the client.

The following subsections detail the technical stack, the responsibilities of each tier, and the defensive engineering decisions integrated into the architecture.

4.1 Overview of the Three-Tier Architecture

As illustrated in Figure 2, the system architecture is divided into three primary components: the Client Tier, the Application Tier, and the Data Tier.

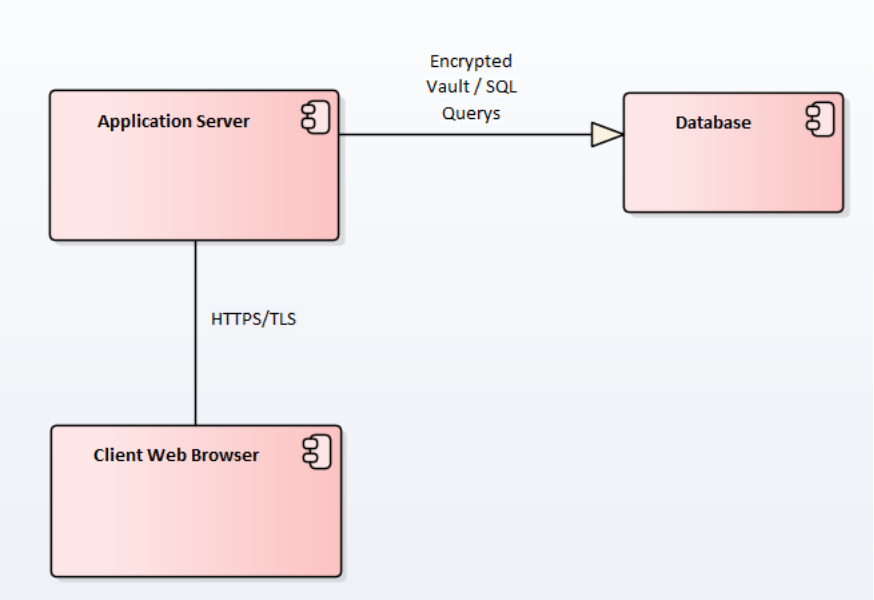


Figure 2. Three-tier architectural overview and communication pathways of the secure password manager.

A. Client Tier (Presentation Layer)

- **Technology & Implementation:** The presentation layer resides entirely within the user's browser, built using standard HTML5, CSS3, and modular JavaScript.
- **Security Responsibilities:** The client tier acts solely as the interface for rendering views and capturing user inputs (such as master credentials and vault entries). To protect data in transit against eavesdropping and man-in-the-middle (MitM) attacks, all communication between the browser and the application server is strictly enforced over **HTTPS utilizing TLS 1.3**. Furthermore, strict Content Security Policies (CSP) will be implemented to mitigate client-side risks such as Cross-Site Scripting (XSS).

B. Application Tier (Business Logic Layer)

- **Technology & Implementation:** The backend application server handles the core business logic, user session management, authentication workflows, and cryptographic operations. It will be implemented using a modern web framework (e.g., Python/Flask or Node.js/Express) chosen for its built-in security features and robust community support.
- **Security Responsibilities:** This tier acts as the primary defense barrier of the application. It is responsible for:
    - **Input Sanitization and Validation:** Enforcing strict server-side whitelisting for all incoming parameters to prevent injection attacks.
    - **Session Management:** Issuing cryptographically secure, randomized session tokens with appropriate security flags (HttpOnly, Secure, SameSite) to prevent session hijacking and CSRF.

    - **Cryptographic Engine:** Executing secure password hashing (such as Argon2 or bcrypt) for master credentials and managing the encryption/decryption keys used to protect individual vault items.

C. Data Tier (Persistent Storage Layer)

- **Technology & Implementation:** The data tier consists of a relational database management system (such as PostgreSQL or SQLite) responsible for persistently storing application data, including user profiles, audit logs, and encrypted user vaults.
- **Security Responsibilities:** The database is isolated from direct public access and can only be queried through parameterized interfaces managed by the application tier, completely eliminating SQL injection vectors.

4.2 Cryptographic Design and Data Protection

A core architectural requirement of a password manager is ensuring that sensitive user data remains confidential even if the database storage is compromised. To achieve this:

- **Zero-Knowledge / Client-Derived Keying Concept:** The server never stores the user's master password in plain text, nor does it possess the raw encryption keys required to decrypt the vault. Master passwords are securely hashed before validation.
- **Data-at-Rest Encryption:** Individual credential records within the vault are encrypted using strong symmetric encryption algorithms (such as AES-256 in Galois/Counter Mode - GCM) before being committed to the database.

4.3 Error Handling and Logging Architecture

To comply with defensive programming standards and prevent information disclosure:

- **Generic Error Messages:** The application tier intercepts internal exceptions (such as database connection failures or parsing errors) and returns generic, sanitized error messages to the user interface, preventing stack trace leakage.
- **Security Logging:** Critical security events—including successful logins, failed authentication attempts, and unauthorized access triggers—are systematically recorded in an internal audit log to support future monitoring and forensic analysis.

5. Threat Model and Vulnerability Mapping

A robust threat model must account for both modern architectural flaws and input-driven attacks. This section evaluates the password manager's defensive posture against the 2025 OWASP Top 10 and specific attack vectors analyzed in the initial phases of this course, specifically focusing on cross-site scripting (XSS), HTML injection, and the circumvention of client-side controls.

5.1 Mapping to Course Attack Classes (Weeks 1–4)

- **Cross-Site Scripting (XSS):** If an attacker successfully submits unsanitized input—such as `<script>alert(document.domain);</script>` or an image tag with a malicious `onerror` event handler—the browser may execute arbitrary JavaScript. This can result in DOM manipulation or, critically, session hijacking if sensitive cookies lack the `HttpOnly` flag.
    - *Mitigation:* The application will employ strict context-aware output encoding and avoid unsafe JavaScript methods (such as `dangerouslySetInnerHTML` in React) to prevent reflected, stored, and DOM-based XSS attacks.
- **HTML Injection and Content Spoofing:** Attackers may attempt to inject unauthorized HTML elements, such as fake `<form>` tags, to mimic legitimate login screens and phish credentials from targets. Furthermore, attackers could manipulate URL parameters to render spoofed warning messages.
    - *Mitigation:* All user-controllable input, including URL parameters, will be strictly sanitized on the server before rendering, neutralizing HTML entities.
- **Bypassing Client-Side Controls:** Client-side validation mechanisms—such as HTML `maxlength` attributes, disabled form elements, and hidden form fields—are strictly for user experience and can be trivially bypassed using browser developer tools or intercepting proxies like Burp Suite.
    - *Mitigation:* The application will operate under the principle that all data transmitted from the client is untrusted. Rigorous server-side validation will be enforced for all requests to prevent parameter tampering and unauthorized actions.

5.2 Mapping to OWASP Top 10 (2025)

- A01:2025 - Broken Access Control & A07:2025 - Authentication Failures: Attackers may attempt to manipulate hidden fields or HTTP cookies to access another user's vault. *Mitigation:* Implementation of cryptographically secure session management and strict server-side authorization checks for every endpoint.

- A04:2025 - Cryptographic Failures: Inadequate encryption could expose the stored passwords. *Mitigation:* Adherence to industry-standard key derivation and symmetric encryption algorithms (detailed in Section 6).
- A05:2025 - Injection: Malicious payloads targeting the backend database. *Mitigation:* Exclusive use of parameterized SQL queries to separate code from data.
- A10:2025 - Mishandling of Exceptional Conditions: Unhandled errors could leak stack traces or system configurations. *Mitigation:* Implementation of a global exception handler that returns generic, safe error messages to the client.

6. Cryptographic Design and Trust Boundaries

The core security of the password manager relies on strict trust boundaries and robust cryptographic protocols, ensuring that a database compromise does not equate to a plaintext credential breach.

- **Master Password and Key Derivation:** The user's master password is the foundation of the cryptographic model. It is never stored in plaintext. Instead, the application utilizes a computationally expensive Key Derivation Function (KDF), such as Argon2id, combined with a unique cryptographic salt.
- **Server-Side Encryption:** The trust boundary dictates that while the client transmits data over a secure channel, the server performs the cryptographic heavy lifting to protect data at rest. When a user submits a new credential to the vault, the server encrypts this payload (the website URL, username, and stored password) using strong symmetric encryption, specifically AES-256 in Galois/Counter Mode (GCM).
- **Database Storage:** The relational database stores only the resulting ciphertext of the vault entries, the randomly generated Initialization Vectors (IVs) required for AES-GCM, the user's cryptographic salt, and the heavily hashed representation of the master password used solely for authentication verification.

7. Authentication and Session Model

To defend against session hijacking, fixation, and unauthorized access, the application will enforce a highly restrictive session management lifecycle.

- **Session Tokens and Cookie Security:** Upon successful authentication, the server generates a high-entropy, randomized session ID. This ID is transmitted to the client via an HTTP cookie. To mitigate the risk of XSS attacks stealing the session, the cookie will be flagged with `HttpOnly`, explicitly forbidding JavaScript access to the cookie value. Additionally, the `Secure` flag will be set, guaranteeing that the browser only transmits the cookie over encrypted HTTPS connections. The

`SameSite=Strict` flag will be applied to prevent Cross-Site Request Forgery (CSRF).

- **Fixation Prevention and Expiry:** To prevent session fixation attacks, the backend will aggressively regenerate the session ID immediately upon user login and privilege changes. Sessions will be strictly governed by both an absolute timeout (e.g., 12 hours) and an idle timeout (e.g., 15 minutes of inactivity), forcing re-authentication upon expiration.
- **Multi-Factor Authentication (MFA):** As an optional but highly recommended layer, users can enable Time-Based One-Time Passwords (TOTP). This ensures that even if a master password is compromised, the attacker cannot establish a valid session without physical access to the user's MFA device.

8. Repository Initialization and Implementation Scope

In accordance with project requirements, a version control repository has been initialized to track the development lifecycle. The repository includes a foundational `README.md` file that serves as the technical entry point for the project.

The initialized repository structure defines:

- **Project Scope:** A clear outline of the application's purpose, threat model, and primary defensive objectives.
- **Planned Routes and API Endpoints:** A documented map of the application's structure, including authentication routes (e.g., `/login`, `/register`, `/logout`), vault management endpoints (e.g., `/api/vault/add`, `/api/vault/retrieve`), and user settings.
- **Feature Roadmap:** A checklist of required security features, including TLS enforcement, AES-256 encryption integration, and input sanitization middleware.
- **Local Development Instructions:** Step-by-step documentation detailing how to securely configure environment variables, initialize the local database schema, and launch the application (e.g., via Docker Compose) for local penetration testing and evaluation.